

**INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY (ICDFA)**  
**SBT-DF203 — Basic Networking Skills for Digital Forensics**  
**Lab 3: SYN Flood Attack Investigation Using TShark**  
Practical Forensic Report

**Practical Report Metadata**

| Field                | Detail                                               |
|----------------------|------------------------------------------------------|
| Student Name         | Mary-Joy Adewole                                     |
| Registration Number  | 2025/FWSD/11468                                      |
| Course Code          | SBT-DF203                                            |
| Course Title         | Basic Networking Skills for Digital Forensics        |
| Instructor           | Aminu Idris, AMCPN — Founder, ICDFA                  |
| Practical Title      | Lab 3: SYN Flood Attack Investigation Using TShark   |
| Delivery Block       | 1/3 of 3 (5–11 September 2026)                       |
| Case/Lab Identifier  | SBT-DF203-Lab3-MaryJoyAdewole                        |
| Analysis Workstation | Kali Linux 2026.2 VM (VMware Workstation), host kali |
| Report Date          | 11 September 2026                                    |

## 1. Executive Summary

This report documents a bounded, authorized investigation into TCP SYN activity directed at a local training Apache service running on 127.0.0.1:80 inside an isolated Kali Linux VM. The exercise established a normal three-way-handshake baseline, generated a strictly bounded four-packet SYN simulation using Scapy, and used tshark to extract, quantify and interpret SYN, SYN-ACK, ACK and RST indicators from the resulting capture.

The bounded simulation produced four initial SYN packets from four distinct ephemeral source ports, each answered by a SYN-ACK from the Apache service, and each subsequently closed by an RST rather than a completed handshake. Wireshark's expert analysis additionally flagged the crafted SYNs as missing the SACK\_PERM TCP option, a marker consistent with synthetically generated (non-stack-native) packets. No retransmissions, lost segments or duplicate ACKs were observed. These findings reproduce the packet-level signature of a SYN flood but, being four packets against localhost within roughly 0.2 seconds, do not by themselves constitute evidence of denial of service against the Apache service.

## 2. Authority, Scope and Safety Controls

This practical was carried out under the authority of the SBT-DF203 Lab 3 manual (Aminu Idris, ICDFA), entirely within an isolated Kali Linux 2026.2 virtual machine running on VMware Workstation. No third-party systems, production networks or public networks were targeted.

- Target service: local Apache 2.4 instance bound to 127.0.0.1:80 only.
- Simulation scope: exactly four SYN packets sent via a fixed Scapy script (COUNT = 4), against 127.0.0.1:80 only, with no continuous automation or address spoofing.
- Capture scope: loopback interface (lo) only, bounded by an explicit 20-packet auto-stop.
- Evidence handling: original captures preserved under evidence/, analysis performed on hashed working copies under working/, SHA-256 recorded at each preservation step.

```

kali@kali: ~$ sudo apt update
Get:1 https://kali.download/kali kali-rolling InRelease [31.8 kB]
Get:2 https://kali.download/kali kali-rolling/main amd64 Packages [21.5 MB]
Get:3 https://kali.download/kali kali-rolling/main amd64 Contents [53.7 MB]
Fetched 62.2 MB in 1min 28s (520 kB/s)
1248 packages can be upgraded. Run 'apt list --upgradable' to see them.

kali@kali: ~$ sudo apt install tshark wireshark python3-scapy
tshark is already the newest version (4.6.0-1).
wireshark is already the newest version (4.6.0-1).
python3-scapy is already the newest version (2.7.0+dfsg1-1).
python3-scapy set to manually installed.
Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1248
* apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Fri 2026-09-11 03:26:31 EDT; 1h 6min ago
   Invocation: c3ae91527fca2abab4c78302045
   Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 666 (apache2)
   Status: "Total requests: 7; Idle/Busy workers 100/0;Requests/sec: 0.00177; Bytes served/sec: 54 B/sec"
   Tasks: 7 (limit: 2048)
   Memory: 20M (peak: 37M, swap: 9.1M, swap peak: 9.1M)
   CPU: 1.023s
   Group: /system.slice/apache2.service
   ExecStart: /usr/sbin/apache2 -k start -DFOREGROUND
   ExecStart: /usr/sbin/apache2 -k start -DFOREGROUND
   ExecStart: /usr/sbin/apache2 -k start -DFOREGROUND
   ExecStart: /usr/sbin/apache2 -k start -DFOREGROUND
   ExecStart: /usr/sbin/apache2 -k start -DFOREGROUND
   ExecStart: /usr/sbin/apache2 -k start -DFOREGROUND

Sep 11 03:26:38 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 11 03:26:31 kali apache2[666]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1. Set the 'ServerName' directive globally to suppress this message
Sep 11 03:26:31 kali systemd[1]: Started apache2.service - The Apache HTTP Server.

kali@kali: ~$ systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Fri 2026-09-11 03:26:31 EDT; 1h 6min ago
   Invocation: c3ae91527fca2abab4c78302045
   Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 666 (apache2)
   Status: "Total requests: 7; Idle/Busy workers 100/0;Requests/sec: 0.00177; Bytes served/sec: 54 B/sec"
   Tasks: 7 (limit: 2048)
   Memory: 20M (peak: 37M, swap: 9.1M, swap peak: 9.1M)
   CPU: 1.023s
   Group: /system.slice/apache2.service
   ExecStart: /usr/sbin/apache2 -k start -DFOREGROUND
   ExecStart: /usr/sbin/apache2 -k start -DFOREGROUND
   ExecStart: /usr/sbin/apache2 -k start -DFOREGROUND
   ExecStart: /usr/sbin/apache2 -k start -DFOREGROUND
   ExecStart: /usr/sbin/apache2 -k start -DFOREGROUND
   ExecStart: /usr/sbin/apache2 -k start -DFOREGROUND

```

Figure 1: Environment setup — apache2, tshark, wireshark and python3-scapy installed and apache2.service enabled and started.

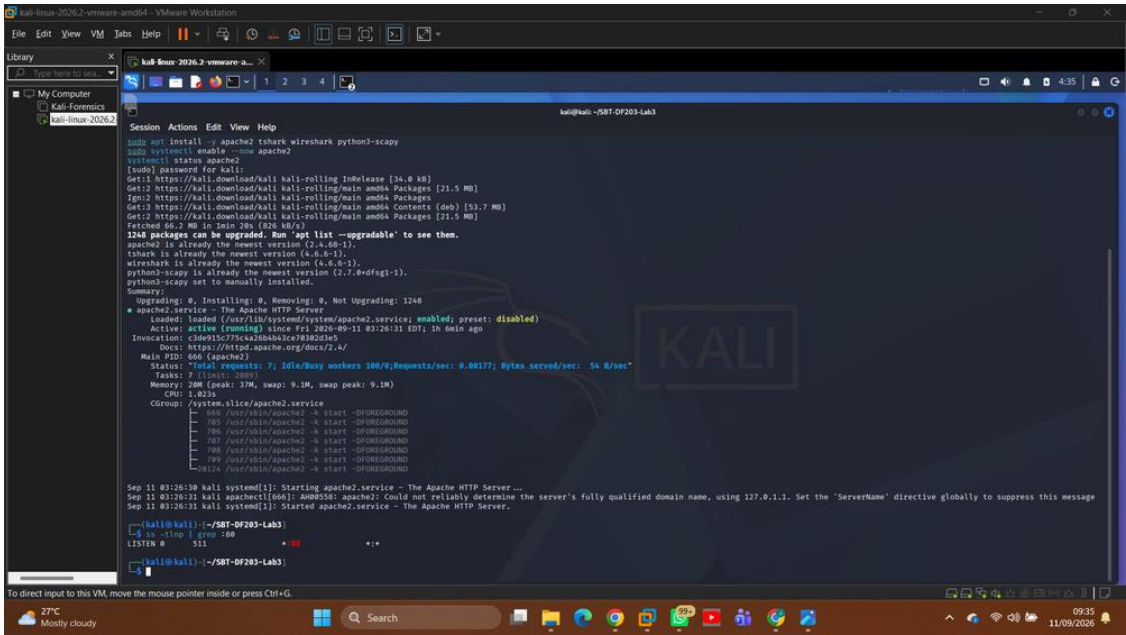


Figure 2: Apache service confirmed active (running) and listening on TCP port 80 (ss -tlnp).

### 3. Evidence Description and Hashes

Four capture/evidence artifacts were generated or retrieved during this practical. SHA-256 hashes were calculated at the point of preservation and again on the working copy used for analysis, confirming an unbroken chain of custody.

| Evidence File                       | Source / Generation Method                                                | SHA-256                                                                                    |
|-------------------------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| mySYNFloodCapture.pcap              | Authorized training resource (frankwxu/digital-forensics-lab GitHub repo) | 14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5                           |
| normal_http.pcapng                  | tshark loopback capture during baseline curl request                      | (recorded in reports/normal_handshake_flags.tsv — not separately hashed per manual Part A) |
| bounded_syn_activity.pcapng         | tshark loopback capture during bounded 4-packet Scapy simulation          | 8b78b45b45171dd4b15055e39441a5759fbafc43a166cdb5687a150f6afe9c5                            |
| bounded_syn_activity_working.pcapng | Working copy of bounded_syn_activity.pcapng (cp --preserve=timestamps)    | 8b78b45b45171dd4b15055e39441a5759fbafc43a166cdb5687a150f6afe9c5 (matches original)         |

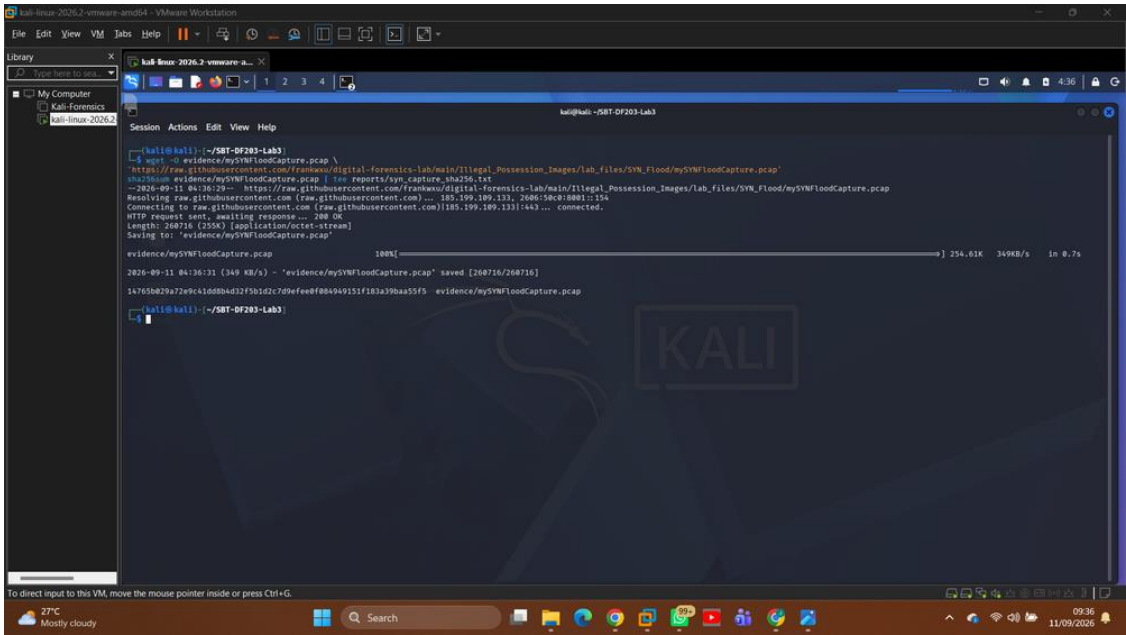


Figure 3: Download and SHA-256 hash of the authorized training PCAP (mySYNFloodCapture.pcap).

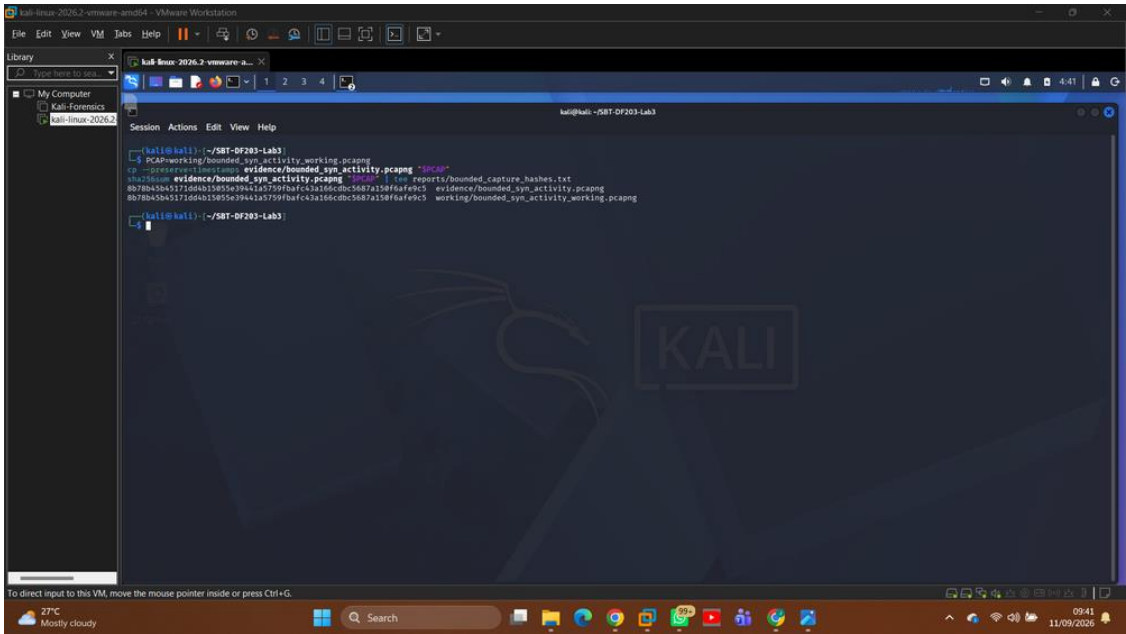


Figure 4: SHA-256 hashes of the bounded capture and its working copy, confirming an identical match.

Mini Chain-of-Custody Worksheet

| Field                       | Entry                                                                                                                        |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Case/lab identifier         | SBT-DF203-Lab3-MaryJoyAdewole                                                                                                |
| Trainee name                | Mary-Joy Adewole                                                                                                             |
| Date and time started       | 11 September 2026, approx. 09:34 AM WAT                                                                                      |
| Evidence file name(s)       | mySYNFloodCapture.pcap;<br>normal_http.pcapng;<br>bounded_syn_activity.pcapng                                                |
| Source or generation method | See evidence table above                                                                                                     |
| Original SHA-256            | See evidence table above                                                                                                     |
| Working-copy SHA-256        | 8b78b45b45171dd4b15055e39441a5759fbafc43a166cdcb5687a150f6afe9c5<br>(bounded_syn_activity_working.pcapng) — matches original |

| Field                   | Entry                                                                                                 |
|-------------------------|-------------------------------------------------------------------------------------------------------|
| Analysis workstation/VM | Kali Linux 2026.2, VMware Workstation, host "kali"                                                    |
| Notes on any changes    | None. Original captures preserved under evidence/; all analysis performed on the hashed working copy. |

#### 4. Normal Handshake Baseline

A 15-second loopback capture was taken on tcp port 80 while issuing a single curl request to <http://127.0.0.1/>. The extracted SYN/FIN-flagged frames show a textbook complete handshake and clean connection teardown:

| Frame | Time          | Src:Port        | Dst:Port        | Flags            |
|-------|---------------|-----------------|-----------------|------------------|
| 1     | 04:37:30.2244 | 127.0.0.1:55938 | 127.0.0.1:80    | 0x0002 (SYN)     |
| 2     | 04:37:30.2245 | 127.0.0.1:80    | 127.0.0.1:55938 | 0x0012 (SYN,ACK) |
| 8     | 04:37:30.2624 | 127.0.0.1:55938 | 127.0.0.1:80    | 0x0011 (FIN,ACK) |
| 9     | 04:37:30.2625 | 127.0.0.1:80    | 127.0.0.1:55938 | 0x0011 (FIN,ACK) |

```

kali@kali:~/SBT-DF203-Lab3$ tshark -i lo -f 'tcp port 80' -a duration:15 -w evidence/normal_http.pcapng
[!] 41620
Capturing on 'Loopback: lo'
  # Total  # Received  # Xferd  Average Speed  Time  Time  Time  Current
  #      #      #      #              #      #      #      #
100 10703 100 10703 0 0 268.5k 0
[!] done
tshark -i lo -f 'tcp port 80' -a duration:15 -w evidence/normal_http.pcapng

kali@kali:~/SBT-DF203-Lab3$ tshark -r evidence/normal_http.pcapng -Y 'tcp.flags.syn==1 || tcp.flags.fin==1' -T fields \
  --frame.number -- frame.time -- ip.src -- tcp.srcport -- ip.dst -- tcp.dstport -- tcp.flags \
  | tee reports/normal_handshake_flags.txt
1 2026-09-11T04:13:37.30.224409572-0400 127.0.0.1 55938 127.0.0.1 80 0x0002
2 2026-09-11T04:13:37.30.22451793-0400 127.0.0.1 80 127.0.0.1 55938 0x0012
8 2026-09-11T04:13:37.30.262459791-0400 127.0.0.1 55938 127.0.0.1 80 0x0011
9 2026-09-11T04:13:37.30.262519800-0400 127.0.0.1 80 127.0.0.1 55938 0x0011

```

Figure 5: tshark extraction of the normal HTTP handshake — clean SYN → SYN-ACK → FIN,ACK sequence on a single source port.

#### 5. Suspicious-Traffic Indicators and Timeline

The bounded simulation script (scripts/syn\_probe\_lab.py) sends exactly four crafted SYN packets from random ephemeral source ports to 127.0.0.1:80 and exits.

```

from scapy.all import IP, TCP, RandShort, send

TARGET = '127.0.0.1'
PORT = 80
COUNT = 4

packets = [IP(dst=TARGET)/TCP(sport=RandShort(), dport=PORT, flags='S') for _ in range(COUNT)]
send(packets, verbose=False)
print(f'Sent {COUNT} authorized training SYN packets to {TARGET}:{PORT}')

```

The screenshot shows a VMware Workstation interface with a Kali Linux 2020.2 virtual machine. The terminal window displays the following commands and output:

```

kali@kali: ~/SMT-DF203-Lab3
[kali@kali] ~/SMT-DF203-Lab3
$ nslookup -i 10.10.10.10 -s 20 -w evidence/bounded_syn_activity.pcapng
Capturing on 'loophack': 10
$
[ kali@kali] ~/SMT-DF203-Lab3
$ sudo python3 scripts/syn_probe_lab.py
[sudo] password for kali:
Sent & authorized training SYN packets to 127.0.0.1:80
[ kali@kali] ~/SMT-DF203-Lab3
$

```

The background of the terminal window features a large, faint watermark of the Kali Linux logo.

Page 6

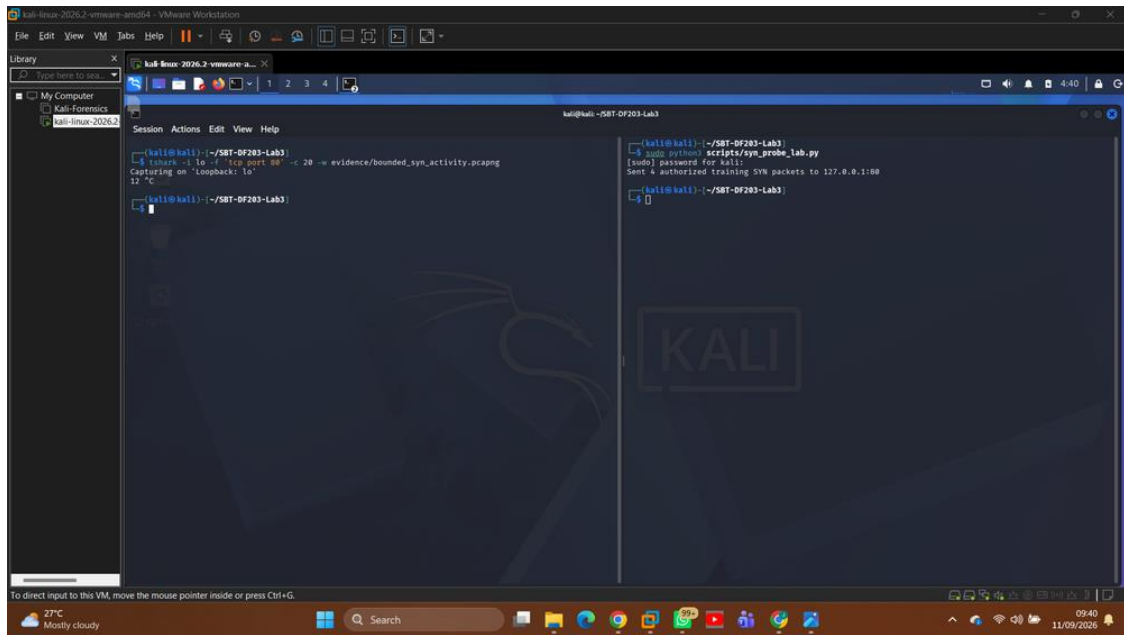


Figure 8: Capture concluded at 12 packets (4 SYN + 4 SYN-ACK + 4 RST) — within the 20-packet auto-stop bound.

### Forensic Timeline (epoch time, relative to first SYN)

| Frame | Time (epoch)      | Src Port             | Event                   |
|-------|-------------------|----------------------|-------------------------|
| 1     | 1789115998.635880 | 62986                | Initial SYN             |
| 2     | 1789115998.635896 | 62986 (server resp.) | SYN-ACK response        |
| 3     | 1789115998.635901 | 62986                | RST (no final ACK sent) |
| 4     | 1789115998.637145 | 52714                | Initial SYN             |
| 5     | 1789115998.637163 | 52714 (server resp.) | SYN-ACK response        |
| 6     | 1789115998.637168 | 52714                | RST                     |
| 7     | 1789115998.638499 | 26475                | Initial SYN             |
| 8     | 1789115998.638516 | 26475 (server resp.) | SYN-ACK response        |
| 9     | 1789115998.638521 | 26475                | RST                     |
| 10    | 1789115998.640046 | 22080                | Initial SYN             |
| 11    | 1789115998.640062 | 22080 (server resp.) | SYN-ACK response        |
| 12    | 1789115998.640067 | 22080                | RST                     |

Every one of the four half-open attempts followed an identical SYN → SYN-ACK → RST pattern within a sub-millisecond window per pair, and the entire 12-frame sequence completed in under 0.005 seconds. No final ACK was ever sent to complete a handshake.

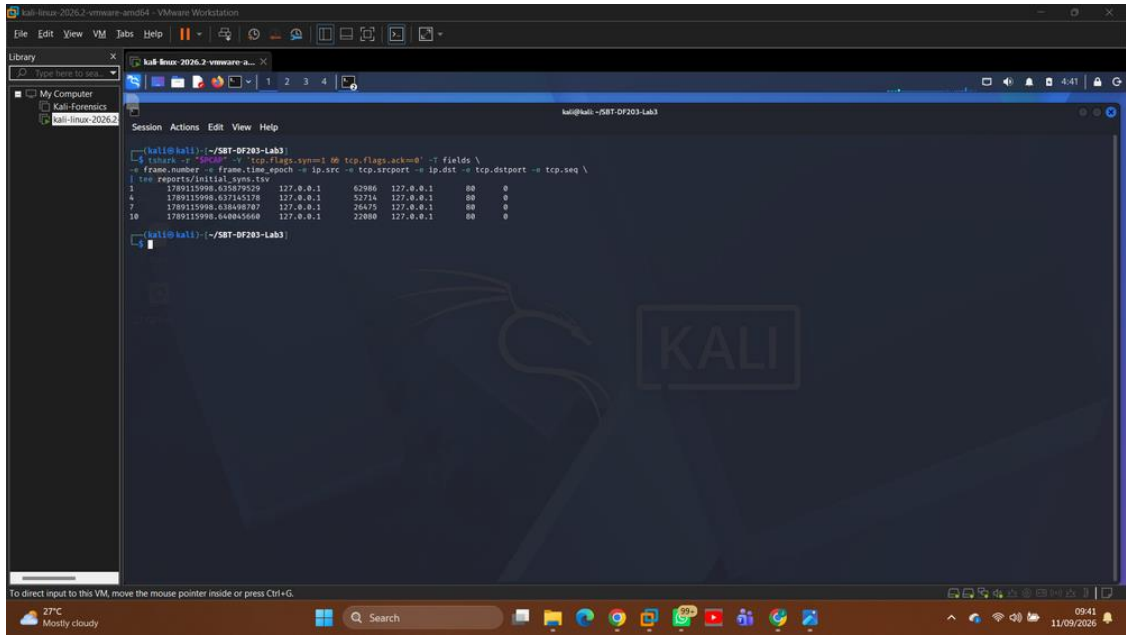


Figure 9: Initial SYN packets ( $tcp.flags.syn==1$  &&  $tcp.flags.ack==0$ ) — four SYNs, four distinct source ports.

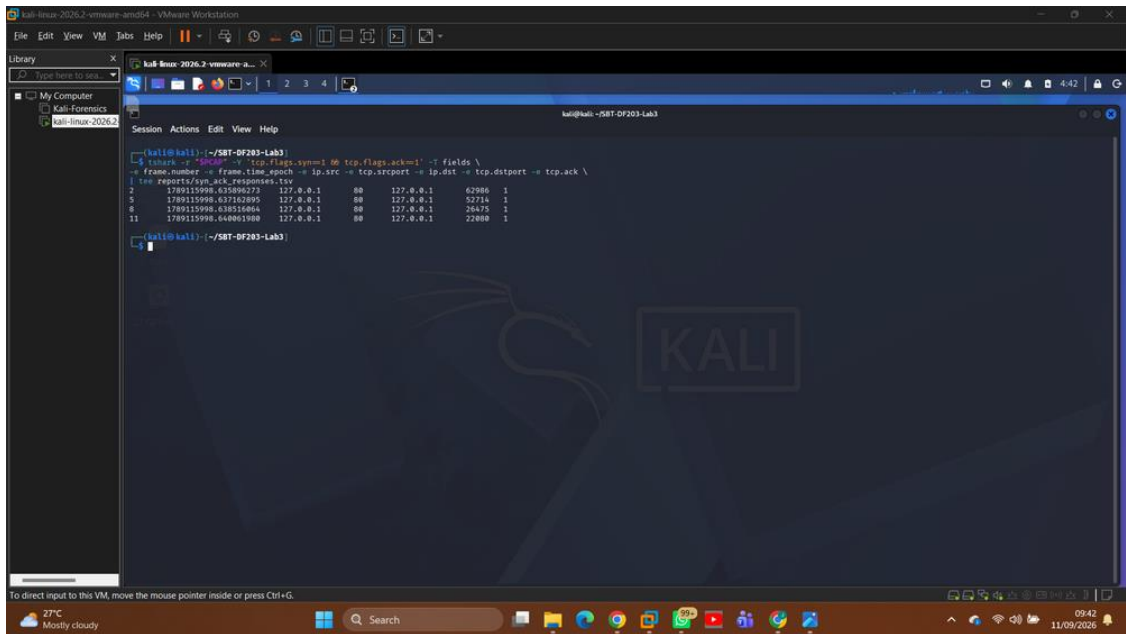


Figure 10: SYN-ACK responses ( $tcp.flags.syn==1$  &&  $tcp.flags.ack==1$ ) — Apache answered all four SYNs.

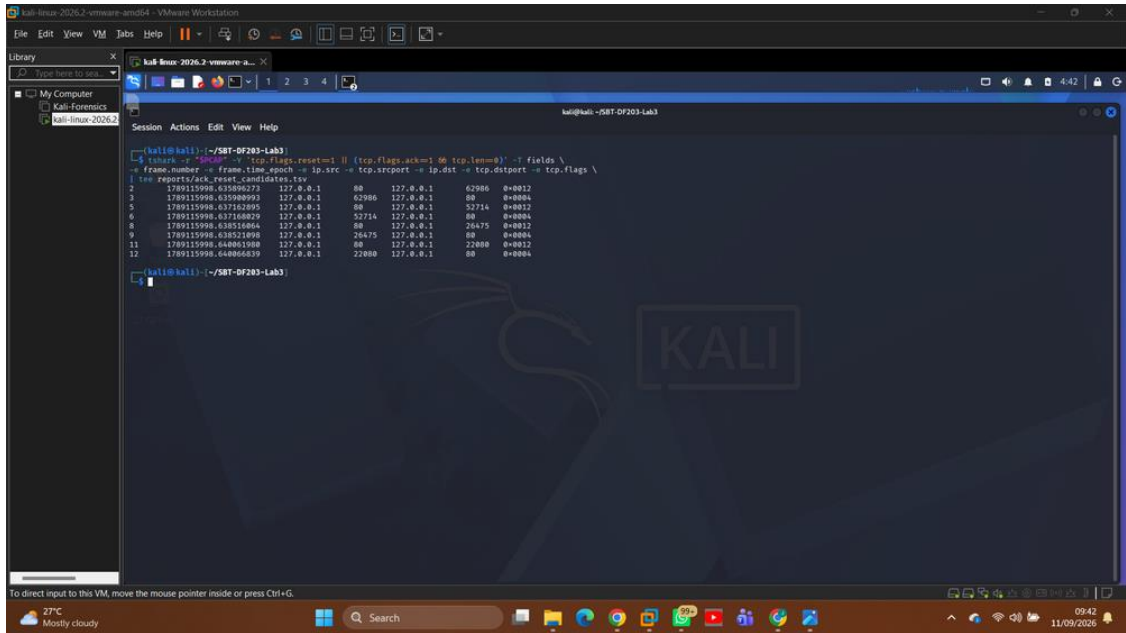


Figure 11: RST and ACK candidate frames — every SYN-ACK was closed with an RST, never a completed handshake ACK.

## 6. Quantitative Analysis

| Metric                                             | Value                                               |
|----------------------------------------------------|-----------------------------------------------------|
| Source/Destination pair (127.0.0.1 → 127.0.0.1:80) | 4 initial SYNs                                      |
| Unique client source ports                         | 4 (22080, 26475, 52714, 62986)                      |
| SYN-ACK responses                                  | 4 (100% response rate)                              |
| Completed handshakes (final ACK)                   | 0                                                   |
| RST packets                                        | 4                                                   |
| Retransmissions / lost segments / duplicate ACKs   | 0                                                   |
| Wireshark expert notes                             | 4× "SYN packet does not contain a SACK_PERM option" |

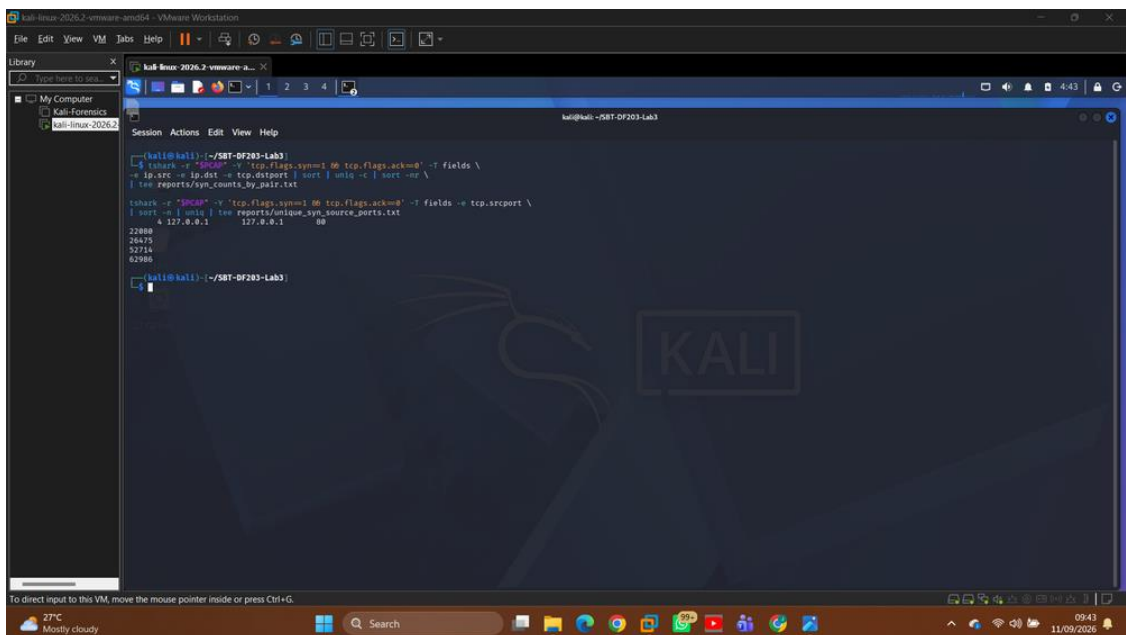


Figure 12: SYN counts by source/destination pair and list of unique client source ports.

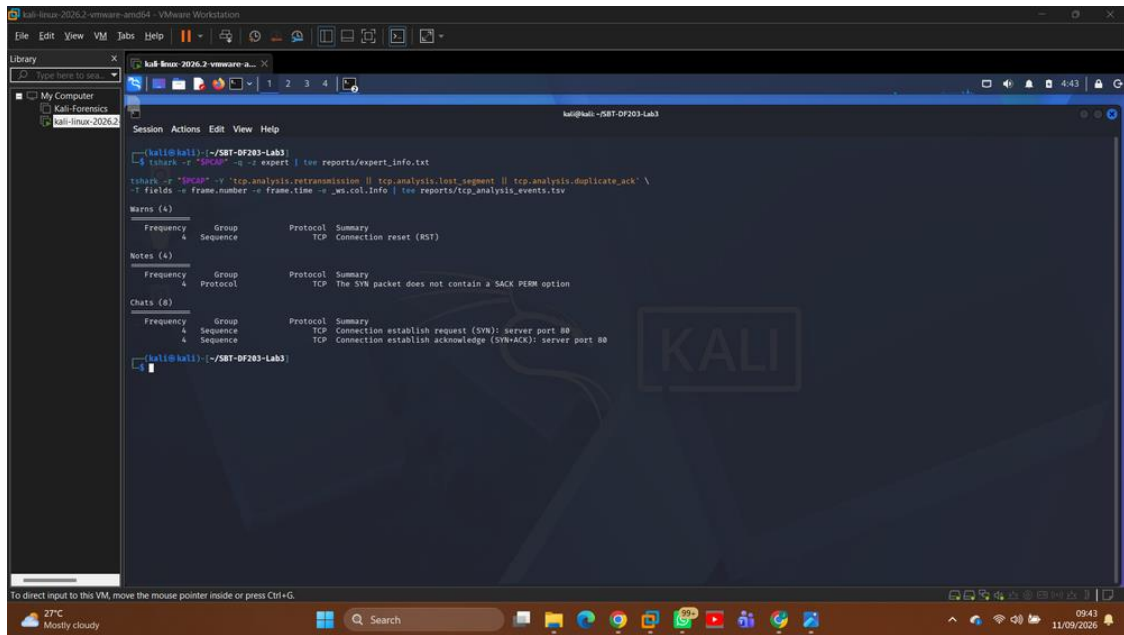


Figure 13: tshark expert information — 4 RST warnings, 4 missing-SACK\_PERM notes, 8 handshake chat entries; no retransmission or duplicate-ACK anomalies.

## 7. Assessment: Packet Pattern vs. Proof of Denial of Service

| Indicator                  | Normal HTTP Session                    | Bounded SYN Activity             | Forensic Meaning                                                      |
|----------------------------|----------------------------------------|----------------------------------|-----------------------------------------------------------------------|
| Initial SYN count          | 1                                      | 4                                | Baseline single connection vs. 4 synthetic SYNs                       |
| SYN-ACK count              | 1                                      | 4                                | Server answered every SYN in both cases                               |
| Completed handshakes       | 1 (implied by successful HTTP request) | 0                                | Bounded set never sent a final ACK; closed via RST instead            |
| Unique client source ports | 1 (55938)                              | 4 (22080, 26475, 52714, 62986)   | Multiple ephemeral ports characteristic of a crafted burst            |
| HTTP request present?      | Yes                                    | No                               | Bounded activity carried no application-layer data — SYN-only probing |
| RST packets                | 0                                      | 4                                | One RST per unanswered handshake                                      |
| Observed duration          | ≈15 s (curl + capture window)          | ≈0.005 s (first SYN to last RST) | Bounded burst vs. natural human-paced session                         |

The bounded capture reproduces the packet-level signature commonly associated with a SYN flood: multiple initial SYNs from distinct source ports, a high SYN-to-completed-handshake ratio (4:0), and RST-terminated half-open attempts. However, a real SYN flood is defined by scale, rate, persistence and measurable service impact — none of which are present here. Four packets delivered in roughly five milliseconds against a service's own loopback address, with Apache continuing to serve requests normally throughout, does not constitute evidence of denial of service. It is, by design, a controlled reproduction of the indicators an analyst would look for, not an attack.

What would be required to escalate this from "suspicious pattern" to "probable SYN flood": sustained elevated SYN-to-completed-handshake ratios over time, evidence of SYN backlog exhaustion or increased connection latency on the target service, corroborating logs from the web server/firewall/load balancer, and traffic volume or duration inconsistent with legitimate client behaviour.

## 8. Detection and Mitigation Recommendations

- Alert on elevated SYN-to-completed-handshake ratios per source and destination, rather than on raw SYN volume alone.
- Monitor SYN backlog exhaustion, retransmission rates, connection-state table size and service latency in real time.
- Enable SYN cookies, tune backlog/timeout parameters sensibly, and apply upstream rate limiting or DDoS protection where the service is internet-facing.
- Retain full packet capture or flow telemetry during suspected incidents, and keep system clocks synchronized (NTP) to support accurate timelines.
- Correlate packet-level evidence with web server, firewall, load-balancer and OS logs before drawing conclusions about service impact.

## 9. Appendix: Command Reference and Screenshot Index

### Commands Used

```
mkdir -p ~/SBT-DF203-Lab3/{evidence,working,exported,reports,screenshots,scripts}
sudo apt update && sudo apt install -y apache2 tshark wireshark python3-scapy
sudo systemctl enable --now apache2 && systemctl status apache2 && ss -tlnp | grep :80
wget -O evidence/mySYNFloodCapture.pcap <authorized training resource URL>
sha256sum evidence/mySYNFloodCapture.pcap | tee reports/syn_capture_sha256.txt
sudo tshark -i lo -f 'tcp port 80' -a duration:15 -w evidence/normal_http.pcapng &
curl --no-keepalive http://127.0.0.1/ >/dev/null ; wait
tshark -r evidence/normal_http.pcapng -Y 'tcp.flags.syn==1 || tcp.flags.fin==1' -T fields ... |
tee reports/normal_handshake_flags.tsv
sudo tshark -i lo -f 'tcp port 80' -c 20 -w evidence/bounded_syn_activity.pcapng
sudo python3 scripts/syn_probe_lab.py
cp --preserve=timestamps evidence/bounded_syn_activity.pcapng
working/bounded_syn_activity_working.pcapng
sha256sum evidence/bounded_syn_activity.pcapng working/bounded_syn_activity_working.pcapng | tee
reports/bounded_capture_hashes.txt
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields ... | tee
reports/initial_syns.tsv
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==1' -T fields ... | tee
reports/syn_ack_responses.tsv
tshark -r "$PCAP" -Y 'tcp.flags.reset==1 || (tcp.flags.ack==1 && tcp.len==0)' -T fields ... |
tee reports/ack_reset_candidates.tsv
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields -e ip.src -e ip.dst -e
tcp.dstport | sort | uniq -c | sort -nr
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields -e tcp.srcport | sort -n |
uniq
tshark -r "$PCAP" -q -z expert | tee reports/expert_info.txt
tshark -r "$PCAP" -Y 'tcp.analysis.retransmission || tcp.analysis.lost_segment ||
tcp.analysis.duplicate_ack' -T fields ... | tee reports/tcp_analysis_events.tsv
```

### Screenshot / Figure Index

| Figure | File                 | Checklist Item                                     |
|--------|----------------------|----------------------------------------------------|
| 1      | Screenshot__310_.png | Environment setup / package installation           |
| 2      | Screenshot__311_.png | Apache service and TCP port 80                     |
| 3      | Screenshot__312_.png | Evidence PCAP download and hash                    |
| 4      | Screenshot__317_.png | Bounded capture hash verification                  |
| 5      | Screenshot__313_.png | Normal handshake in baseline capture               |
| 6      | Screenshot__314_.png | Bounded Scapy script (target/count)                |
| 7      | Screenshot__315_.png | Bounded capture running                            |
| 8      | Screenshot__316_.png | Bounded capture stopped (12 packets)               |
| 9      | Screenshot__318_.png | Initial SYN packet list                            |
| 10     | Screenshot__319_.png | SYN-ACK response list                              |
| 11     | Screenshot__320_.png | ACK/RST behaviour                                  |
| 12     | Screenshot__321_.png | Counts by source/destination + unique source ports |
| 13     | Screenshot__322_.png | Expert information / TCP analysis events           |

**Student Declaration:** *I confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations and conclusions.*

**Mary-Joy Adewole — 2025/FWSD/11468**